

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN

CHƯƠNG TRÌNH THỰC TẬP HÈ UET 2026

BÁO CÁO TỔNG KẾT THỰC TẬP

Phát triển ECU chẩn đoán trên STM32 — Giao tiếp CAN, ISO-TP và UDS (ISO 14229-1)

Nhóm 1

Hà Nội, tháng 8 năm 2026

MỤC LỤC

1. Giới thiệu chung	3
1.1. Ứng dụng của hệ thống chẩn đoán UDS/CAN trong thực tế	3
1.2. Mục tiêu và phạm vi của dự án	3
2. Tổng quan project	4
2.1. Nhóm thực hiện và phân công	4
2.2. Công nghệ và công cụ sử dụng	4
2.3. Cấu trúc mã nguồn	4
2.4. Quy trình làm việc theo V-Model	4
3. Yêu cầu (Requirements)	5
3.1. Yêu cầu hệ thống (SYS)	5
3.2. Yêu cầu phần mềm (SWR)	5
4. Thiết kế hệ thống và phần cứng	6
4.1. Kiến trúc phân tầng và lời gọi hàm giữa các tầng	6
4.2. Sơ đồ tuần tự minh họa: đọc VIN	6
4.3. Khởi tạo hệ thống và vòng lặp chính	7
4.4. Thiết kế phần cứng	8
5. Thiết kế chi tiết từng tầng (Detailed Design)	10
5.1. Tầng CanIf	10
5.2. Tầng ISO-TP	11
5.3. Tầng UDS	18
5.4. DID Manager, DTC Manager, App Engine, App Diag	25
6. Demo	28
7. Test Report	31
8. Kết luận	32
8.1. Kết quả đạt được	32
8.2. Đánh giá quá trình làm việc nhóm	32
8.3. Hạn chế kỹ thuật	32
9. Hướng phát triển	34

1. GIỚI THIỆU CHUNG

1.1. Ứng dụng của hệ thống chẩn đoán UDS/CAN trong thực tế

Trên ô tô hiện đại, hàng chục ECU (Electronic Control Unit) điều khiển động cơ, phanh, túi khí, thân xe... giao tiếp với nhau qua mạng CAN (Controller Area Network, ISO 11898). Để bảo dưỡng, sửa chữa và kiểm định, các ECU này bắt buộc phải hỗ trợ một giao thức chẩn đoán chuẩn hóa: UDS - Unified Diagnostic Services (ISO 14229-1), chạy trên nền vận chuyển ISO-TP (ISO 15765-2) để đóng gói các bản tin chẩn đoán dài hơn một CAN frame.

Trong thực tế, thiết bị chẩn đoán (máy scan tại gara, công cụ hãng như VCDS, hoặc cổng OBD-II bắt buộc theo luật tại nhiều thị trường) dùng UDS để: đọc số VIN và phiên bản phần mềm của ECU, đọc dữ liệu sống (tốc độ, vòng tua, nhiệt độ động cơ...), đọc và xóa mã lỗi DTC (Diagnostic Trouble Code) khi đèn Check Engine sáng, chuyển session chẩn đoán để mở khóa các dịch vụ nâng cao, và ra lệnh reset ECU sau khi cập nhật phần mềm. Đây là hạ tầng bắt buộc của toàn bộ ngành bảo dưỡng ô tô hiện nay, không phân biệt hãng xe.

Dự án của nhóm là mô phỏng lại đúng bài toán trên ở quy mô thu nhỏ: một ECU chẩn đoán tự xây dựng từ tầng vật lý (CAN) lên tới tầng ứng dụng (UDS), chạy trên vi điều khiển STM32F103C8, để hiểu và tái hiện cách một ECU thật trên xe đáp ứng yêu cầu chẩn đoán.

1.2. Mục tiêu và phạm vi của dự án

Mục tiêu của nhóm là xây dựng hai node STM32 giao tiếp qua CAN 500 kbps: một node đóng vai trò Tester/Diagnostic Tool (gửi yêu cầu UDS qua UART từ máy tính), một node đóng vai trò ECU (mô phỏng dữ liệu động cơ, trả lời các dịch vụ UDS). Phạm vi kỹ thuật gồm bốn tầng: CanIf (giao tiếp CAN), ISO-TP (đóng gói/phân mảnh bản tin), UDS (xử lý dịch vụ chẩn đoán), và các module ứng dụng (App Engine mô phỏng động cơ, App Diag đóng vai trò tester, DID Manager, DTC Manager).

2. TỔNG QUAN PROJECT

2.1. Nhóm thực hiện và phân công

Thành viên	Mã sinh viên	Đơn vị
Nguyễn Đình Cường	23020786	FPT Software - Nhóm 1
Nguyễn Trường Giang	23020802	FPT Software - Nhóm 1
Bùi Tiến Dũng	23020792	FPT Software - Nhóm 1
Bùi Xuân Thanh	23020883	FPT Software - Nhóm 1

2.2. Công nghệ và công cụ sử dụng

Hạng mục	Lựa chọn của nhóm
Vi điều khiển	STM32F103C8T6 (ARM Cortex-M3), hai board độc lập đóng vai trò Tester và ECU
Kết nối CAN	CAN1 (PA11/PA12), transceiver ngoài, tốc độ 500 kbps, hai điện trở đầu cuối 120 Ω
Thư viện phần mềm	STM32Cube HAL cho dòng STM32F1, mã ứng dụng viết bằng Embedded C
IDE	STM32CubeIDE, VSCODE
Giao tiếp người dùng	UART 115200 baud tới máy tính, dùng làm giao diện gõ lệnh và xem log
Quản lý mã nguồn	Git/GitHub theo luồng Issue - Branch - Pull Request - Review - Merge

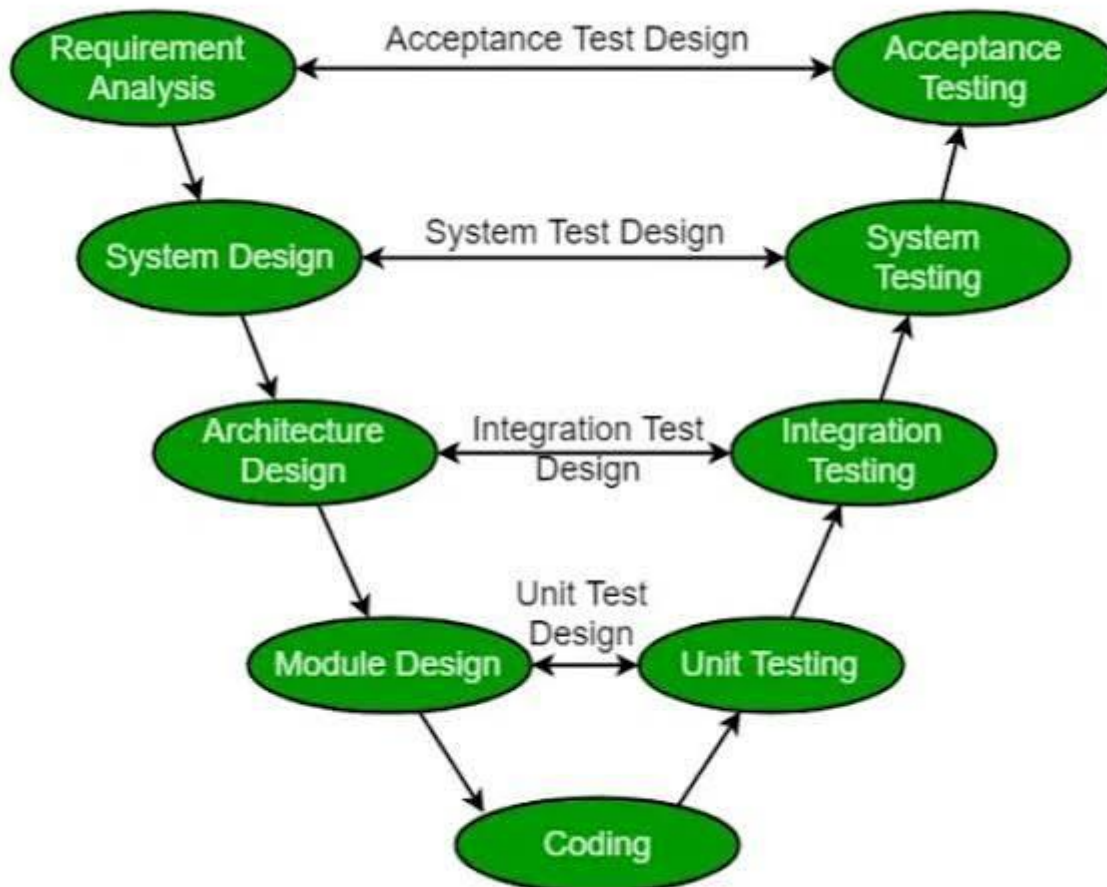
2.3. Cấu trúc mã nguồn

Mã nguồn được tổ chức theo lớp (layered), mỗi tầng là một cặp .c/.h riêng trong Modules/, tách khỏi Core (khởi tạo HAL) và Applications (hai entry point main_diag.c và main_EC.c):

Thư mục	Nội dung
Modules/drivers/canif/	Giao tiếp CAN mức thấp: Can_if.c/.h
Modules/middleware/isotp/	Tầng ISO-TP: isotp.c/.h
Modules/drivers/uds/	Tầng UDS: uds.c/.h
Modules/did/	DID Manager: did_manager.c/.h
Modules/dtc/	DTC Manager: dtc_manager.c/.h
Modules/app/app_engine/	Mô phỏng dữ liệu động cơ phía ECU
Modules/app/app_diag/	Logic tạo request / nhận response phía Tester
Applications/EC/, Applications/Diag/	main_EC.c, main_diag.c - chỉ khởi tạo HAL và vòng lặp chính
docs/	requirements, design, tests, process - tài liệu V-Model của nhóm

2.4. Quy trình làm việc theo V-Model

Nhóm chọn V-Model làm khung quy trình: Yêu cầu hệ thống/phần mềm ở nhánh trái tương ứng với System Test/Acceptance Test ở nhánh phải, Architecture/Module Design tương ứng với Integration Test/Unit Test. Trên lý thuyết đây là quy trình phù hợp cho phần mềm nhúng có ràng buộc an toàn. Mức độ tuân thủ thực tế của nhóm được đánh giá trung thực tại mục 8.2.



Mô hình V-Model: nhánh trái là các bước phân tích - thiết kế, nhánh phải là các mức kiểm thử tương ứng

3. YÊU CẦU (REQUIREMENTS)

Đây là các yêu cầu được nhóm chốt ở đầu dự án, dùng làm gốc cho thiết kế và kiểm thử. Mức độ đạt được thực tế của từng yêu cầu được đối chiếu lại ở chương 7 (Test Report) và chương 8 (Kết luận) - không phải mọi yêu cầu dưới đây đều đã được cài đặt và kiểm chứng đầy đủ.

3.1. Yêu cầu hệ thống (SYS)

Mã	Yêu cầu	Tiêu chí chấp nhận	Kiểm chứng
SYS-001	ECU phát bản tin EngineStatus theo chu kỳ	CAN ID 0x100, DLC 8, chu kỳ 1000 ms $\pm$ 10%	ST-001, AT-001
SYS-002	ECU hoạt động như UDS server trên CAN	Hỗ trợ tối thiểu 5 SID: 0x10, 0x11, 0x19, 0x22, 0x3E	ST-002..009, AT-002
SYS-003	ECU trả VIN qua DID 0xF190	Phản hồi 62 F1 90 + VIN, truyền qua ISO-TP khi dài hơn 7 byte	UT-DID-001, ST-002
SYS-004	ECU trả Software Version qua DID 0xF187	Phản hồi 62 F1 87 + chuỗi phiên bản	ST-003
SYS-005	ECU cung cấp dữ liệu động qua DID	0x0101 VehicleSpeed, 0x0102 RPM, 0x0103 Temp, 0x0104 BatteryVoltage	ST-004, ST-005
SYS-006	ECU quản lý và báo cáo DTC	0x010001 quá nhiệt, 0x010002 điện áp thấp; đọc bằng dịch vụ 0x19	UT-DTC-001/002, ST-010/011
SYS-007	ECU trả negative response cho request không hợp lệ	Định dạng 7F <SID> <NRC>	UT-UDS-002, ST-006
SYS-008	Hệ thống chạy được trên phần cứng thật	STM32 và CAN transceiver hoạt động, có CAN trace hoặc log UART	AT-001..004

3.2. Yêu cầu phần mềm (SWR)

Mã yêu cầu	Nội dung chính	Thành phần
SWR-CAN-001..003	API khởi tạo, truyền frame (kiểm tra NULL/DLC), callback nhận theo ID	CanIf
SWR-ISO-TP-001/002	Single Frame cho payload $\leq$ 7 byte; phân mảnh FF/FC/CF cho payload tới 64 byte	ISO-TP
SWR-UDS-001..003	Định tuyến theo SID; positive = SID+0x40; negative = 7F SID NRC	UDS Dispatcher
SWR-DID-001/002	DID tĩnh 0xF190, 0xF187 và 4 DID động 0x0101..0x0104	DID Manager
SWR-DTC-001	Set DTC active, clear DTC, trả danh sách DTC active	DTC Manager
SWR-APP-001/002	Mô phỏng và đóng gói EngineStatus (speed, RPM, temp, battery)	App Engine

4. THIẾT KẾ HỆ THỐNG VÀ PHẦN CỨNG

4.1. Kiến trúc phân tầng và lời gọi hàm giữa các tầng

Kiến trúc phần mềm gồm 4 tầng xếp dọc, tầng trên gọi hàm public của tầng dưới, tầng dưới gọi callback đã đăng ký để trả dữ liệu lên tầng trên (không có tầng nào gọi ngược lại tầng App trực tiếp):

Kiến trúc phân tầng: APPLICATION/UDS → ISO-TP MIDDLEWARE → CAN IF → STM32 HAL

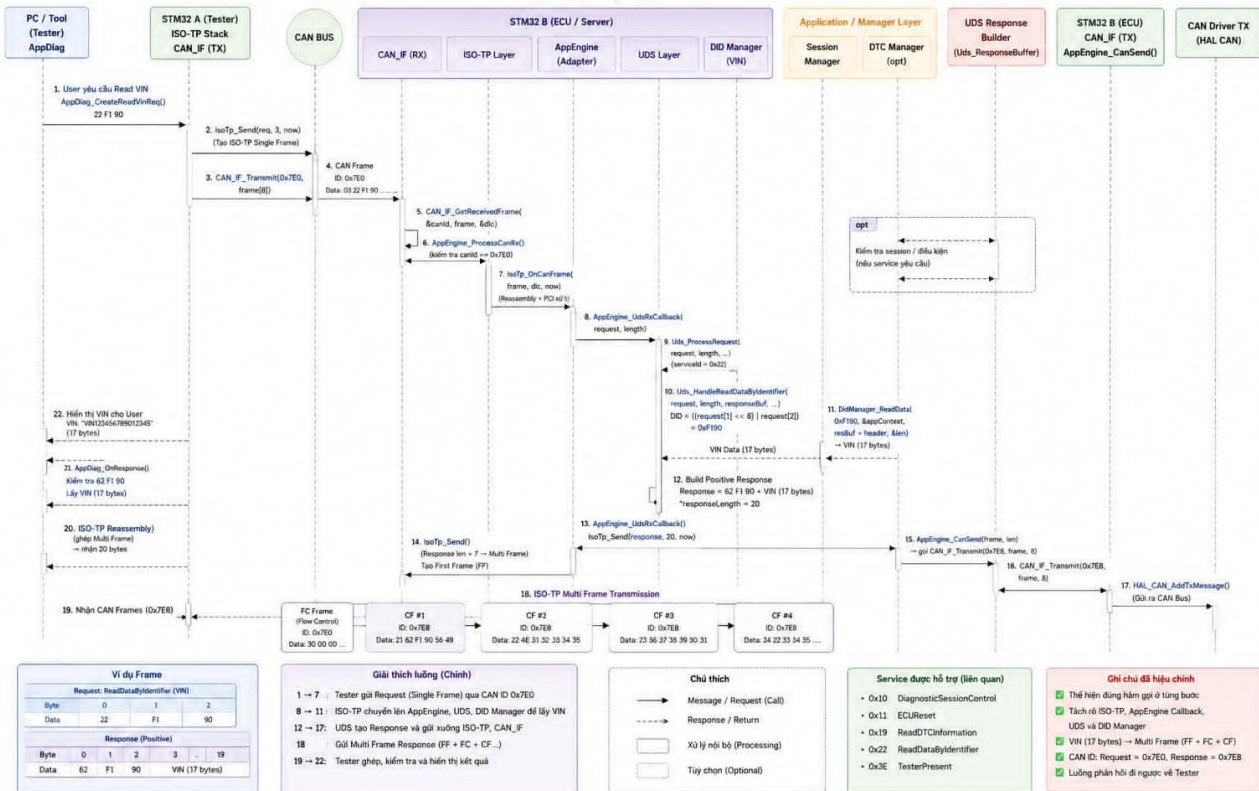
Tầng	Hàm public gọi xuống tầng dưới	Hàm/callback gọi lên tầng trên
APPLICATION / UDS	Uds_Init(), Uds_ProcessRequest() gọi IsoTp_Send() để gửi response	Nhận payload qua rxCallback do ISO-TP gọi
ISO-TP MIDDLEWARE	IsoTp_Send(), IsoTp_OnCanFrame() gọi canSend() (= CAN_IF_Transmit) để gửi frame	Gọi rxCallback (đăng ký khi IsoTp_Init) để đưa payload đã ghép lên UDS
CAN IF	CAN_IF_Transmit() gọi HAL_CAN_AddTxMessage()	CAN_IF_ProcessRxInterrupt() gọi IsoTp_OnCanFrame() khi có frame mới
STM32 HAL / CAN HW	HAL_CAN_AddTxMessage(), HAL_CAN_GetRxMessage()	HAL_CAN_RxFifo1MsgPendingCallback() báo ngắt nhận cho CanIf

Lưu ý quan trọng về liên kết giữa ISO-TP và CanIf: hàm gửi (canSend) và callback nhận (rxCallback) được truyền vào IsoTp_Init() dưới dạng con trỏ hàm, không hard-code tên hàm cụ thể - đây là điểm giúp ISO-TP không phụ thuộc trực tiếp vào CanIf hay UDS, đúng nguyên tắc phân lớp.

4.2. Ví dụ sơ đồ tuần tự minh họa: đọc VIN

Sơ đồ dưới đây là ví dụ minh họa đúng theo thiết kế khi tester đọc VIN (request 22 F1 90): từ CAN driver RX, qua ISO-TP reassembly, UDS dispatcher, DID Manager, tới UDS Response Builder và CAN driver TX. Đây là luồng lý thuyết theo thiết kế; mức độ khớp với hành vi thực tế của code hiện tại được nêu ở chương 5.2 và 5.3.

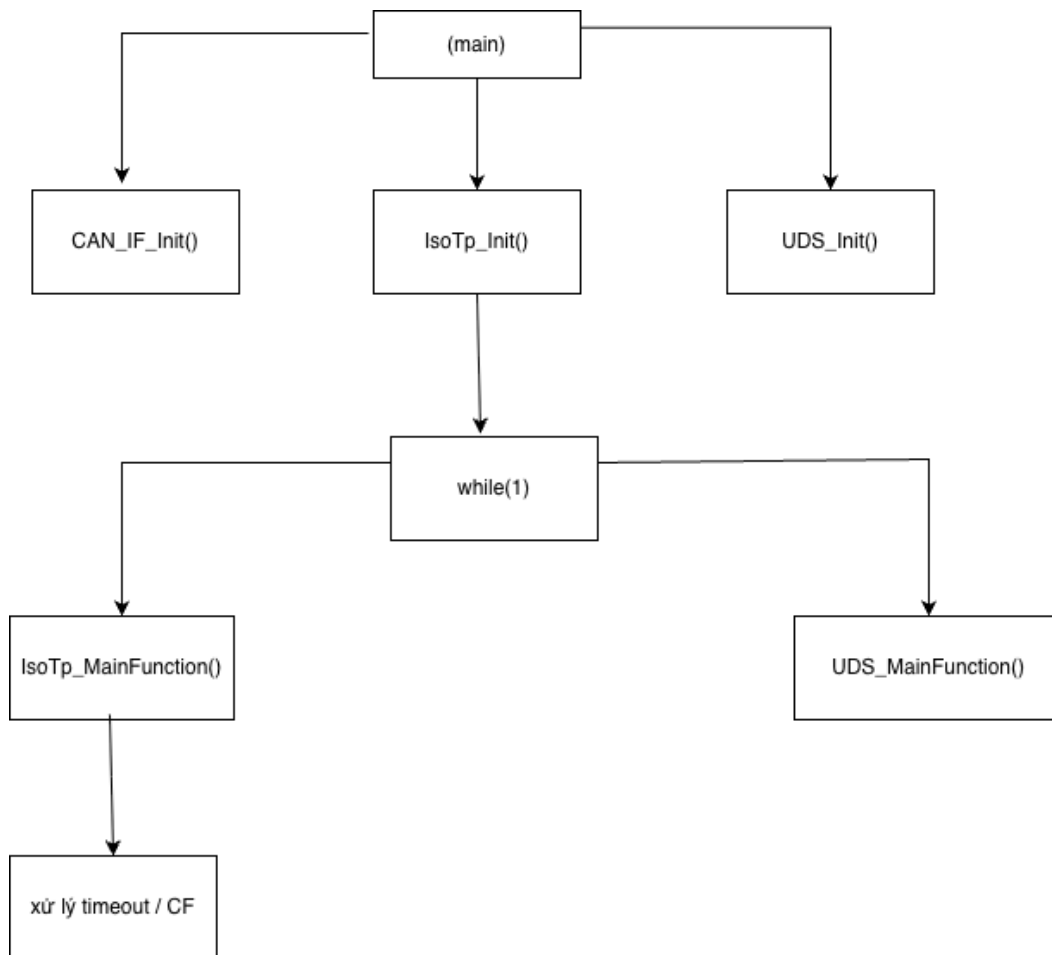
SEQUENCE DIAGRAM: CAN → UDS → CAN (Read VIN 22 F1 90) – ĐÃ HIỆU CHỈNH (VIN = 17 bytes)



Sequence diagram: CAN → ISO-TP → UDS → CAN khi đọc VIN (22 F1 90)

4.3. Khởi tạo hệ thống và vòng lặp chính

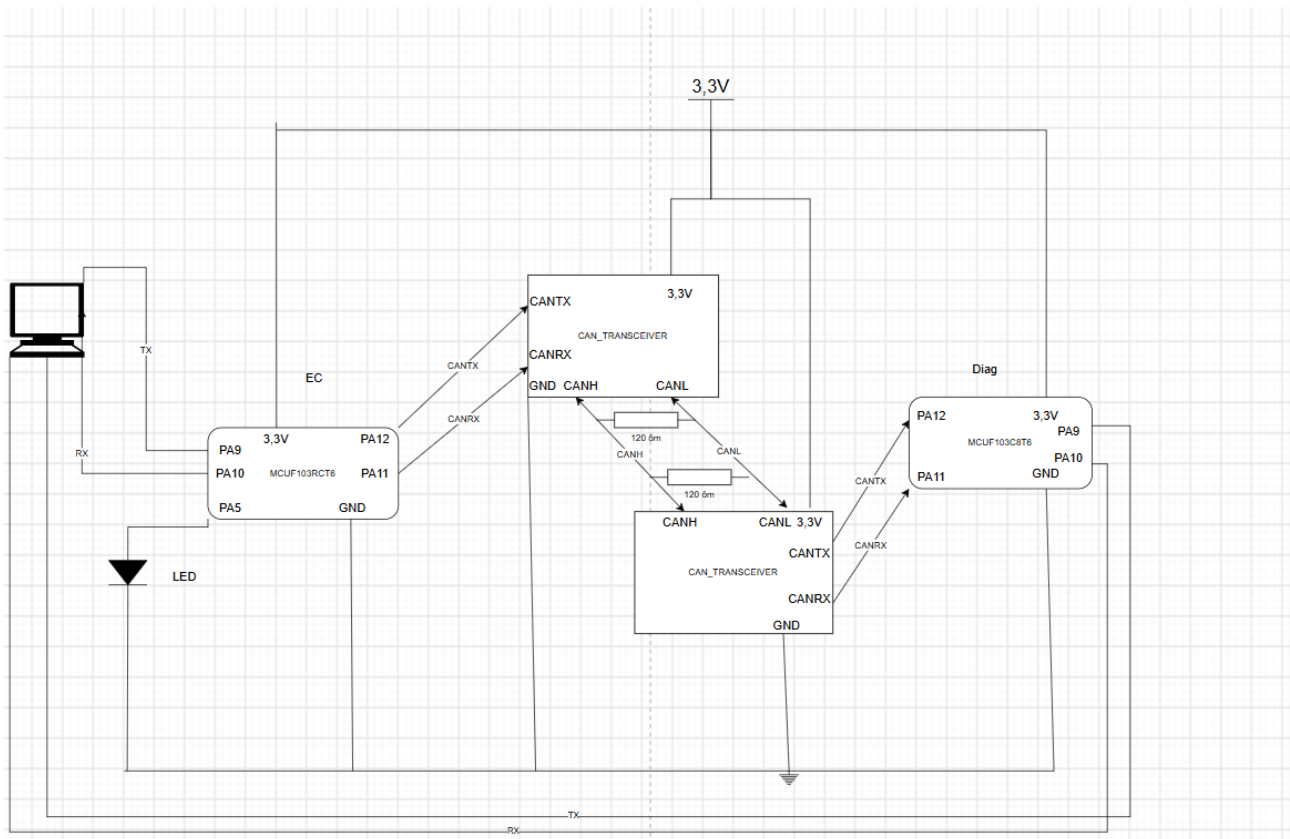
main() ngoài khởi tạo phần cứng thì còn có ba nhiệm vụ chính: gọi CAN_IF_Init(), IsoTp_Init(), UDS_Init(), sau đó vào while(1) gọi tuần tự IsoTp_MainFunction() (xử lý timeout/CF) và UDS_MainFunction.



Khởi tạo hệ thống và vòng lặp chính trong main()

4.4. Thiết kế phần cứng

Hai board STM32F103C8T6 (đặt tên EC và Diag) nối chung một bus CAN qua hai IC transceiver ngoài, có điện trở đầu cuối 120 Ω ở mỗi đầu bus theo đúng chuẩn ISO 11898-2. Board Diag (Tester) nối thêm UART (PA9/PA10) ra máy tính để nhận lệnh và in log, có một LED báo trạng thái trên PA5.



Sơ đồ nguyên lý phần cứng: hai node STM32F103C8T6 nối qua CAN transceiver, UART debug về máy tính

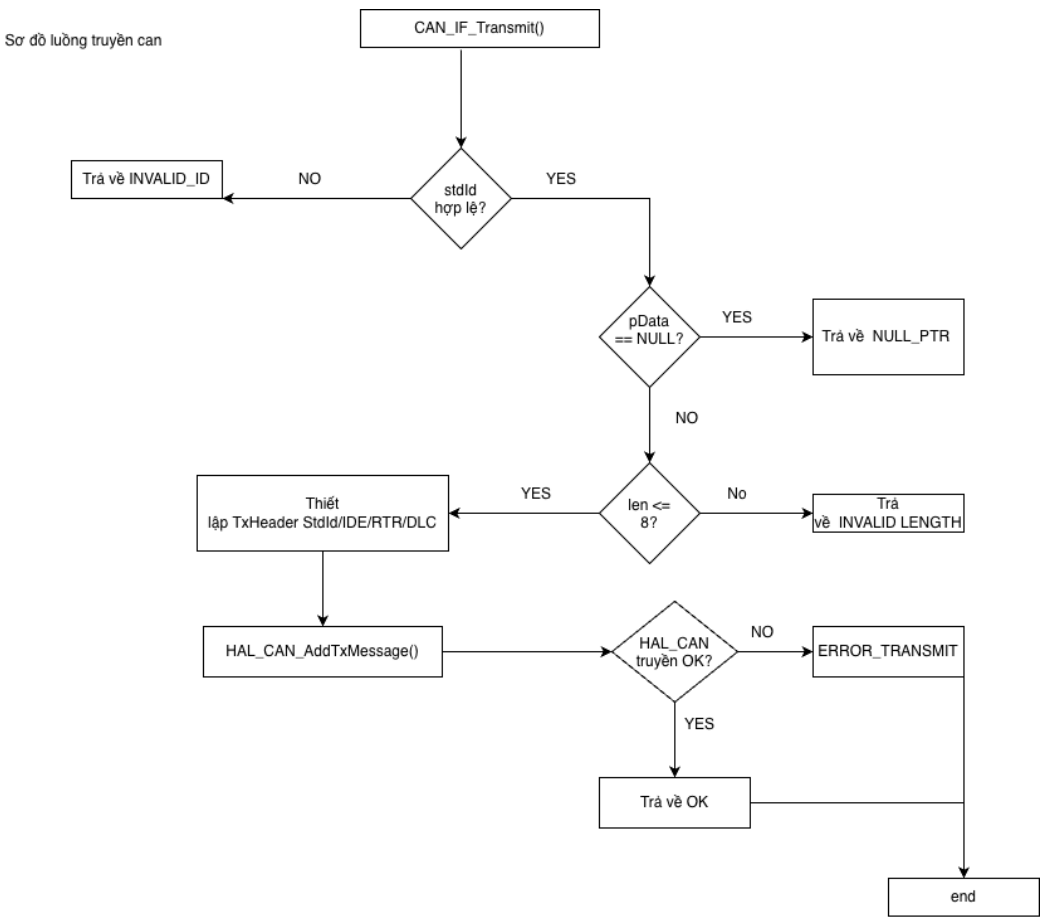
Thông số	Giá trị
Vi điều khiển	STM32F103C8T6 / STM32F103RCT6, Cortex-M3
Chân CAN	PA12 = CAN_TX, PA11 = CAN_RX
Tốc độ CAN	500 kbps, điện trở đầu cuối 120 Ω x2
UART debug (board Diag)	PA9 = TX, PA10 = RX, 115200 baud, 8N1
Nguồn	3.3V cho MCU và transceiver

5. THIẾT KẾ CHI TIẾT TỪNG TẦNG (DETAILED DESIGN)

5.1. Tầng CanIf

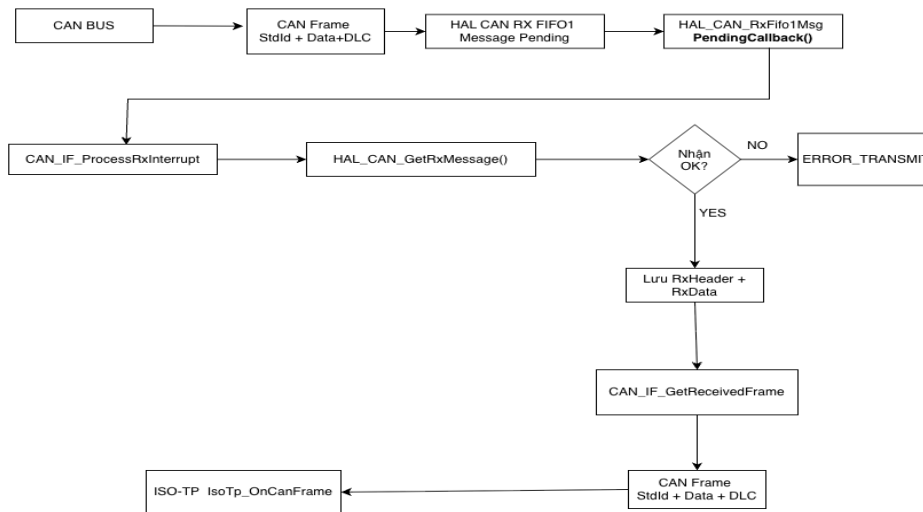
Tầng CanIf chưa có tài liệu detailed design (lưu đồ ở mức từng hàm) như hai tầng ISO-TP và UDS bên dưới - đây cũng là một hạn chế cần ghi nhận. Phần này mô tả bằng lời dựa trên hai lưu đồ luồng tổng thể (TX/RX) và mã nguồn Can_if.c/.h thực tế, là tầng duy nhất của dự án được nhóm xác nhận đã hoàn thiện.

Hàm	Nhiệm vụ
CAN_IF_Init()	Khởi tạo CAN1, cấu hình filter, bật ngắt nhận FIFO1
CAN_IF_Transmit(stdId, pData, len)	Kiểm tra stdId hợp lệ, con trỏ khác NULL, len ≤ 8, rồi gọi HAL_CAN_AddTxMessage()
CAN_IF_GetReceivedFrame(...)	Lấy frame đã nhận kèm ID và DLC thực tế từ bộ đệm nội bộ
CAN_IF_ProcessRxInterrupt()	Được gọi từ HAL_CAN_RxFifo1MsgPendingCallback(), đọc frame và chuyển lên ISO-TP
CAN_IF_HandleTxError()	Đọc bộ đếm lỗi truyền phục vụ chẩn đoán



Luồng truyền CAN frame (CAN_IF_Transmit)

Sơ đồ luồng nhận can

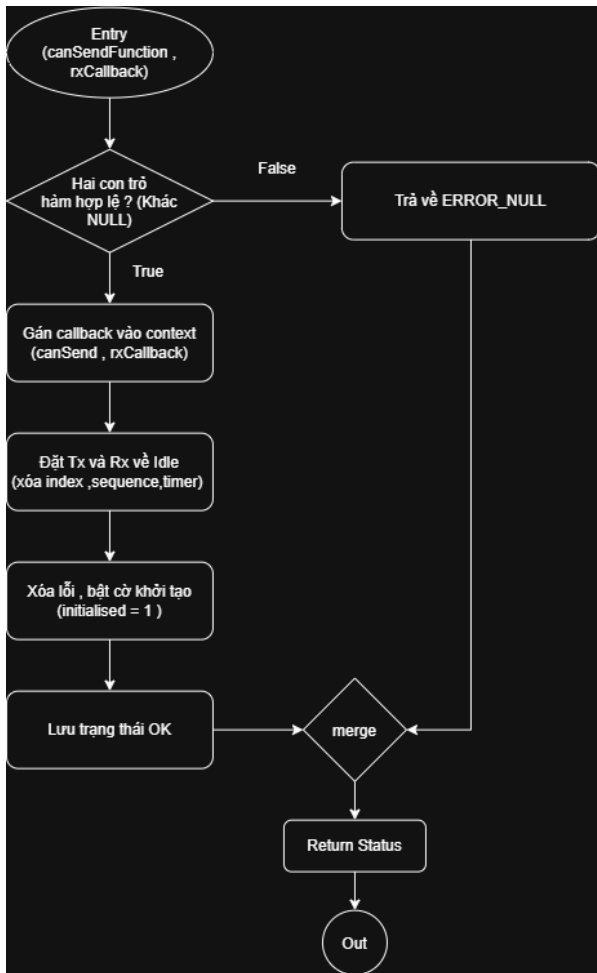


TX

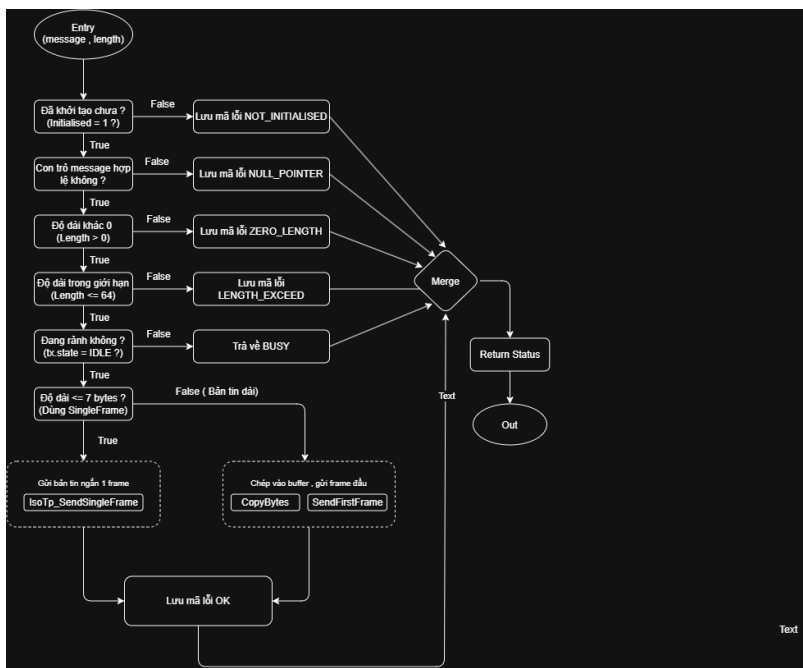
Luồng nhận CAN frame qua ngắt FIFO1

5.2. Tầng ISO-TP

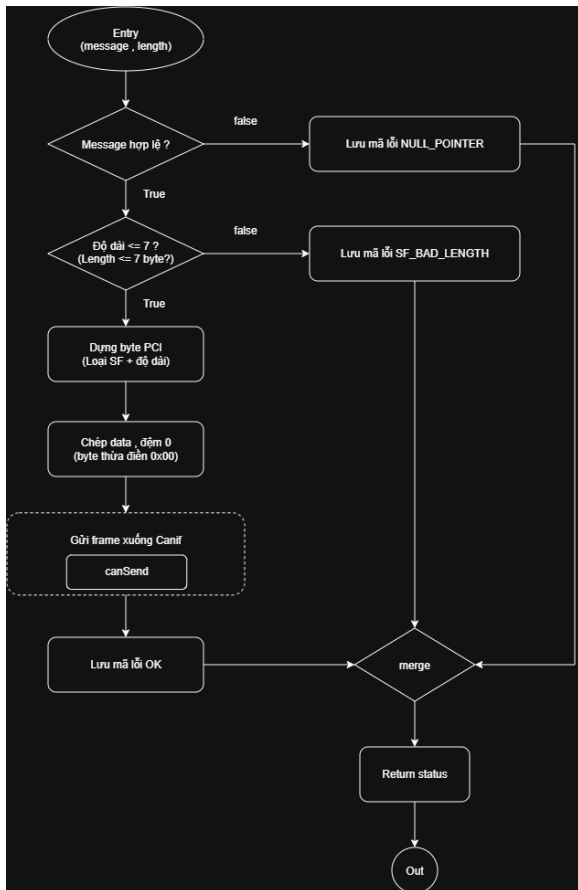
ISO-TP là tầng có tài liệu detailed design đầy đủ nhất của dự án - mỗi hàm trong isotp.c đều có lưu đồ riêng, thiết kế đúng theo ISO 15765-2 với đầy đủ bốn loại frame (SF, FF, CF, FC) và cơ chế Flow Control hai chiều.



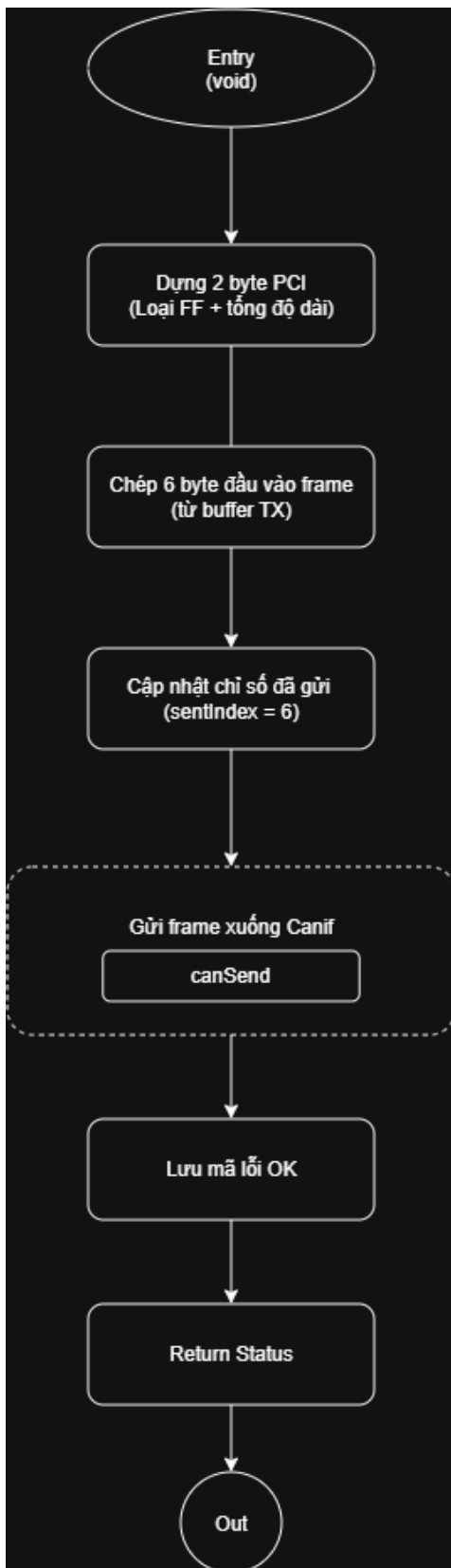
IsoTp_Init: gán callback, đưa TX/RX về Idle



IsoTp_Send: chọn Single Frame hoặc First Frame theo độ dài

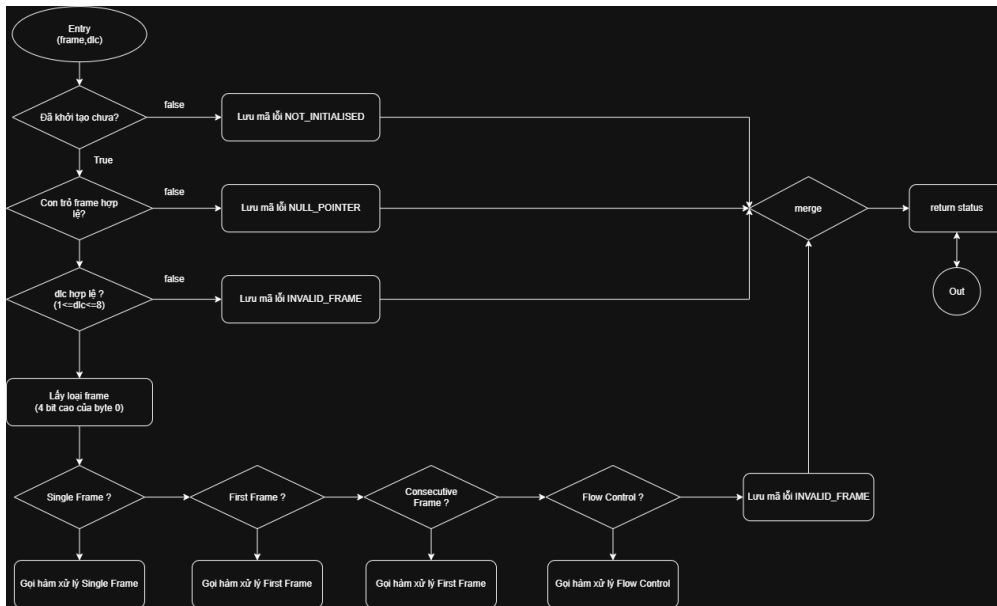


IsoTp_SendSingleFrame

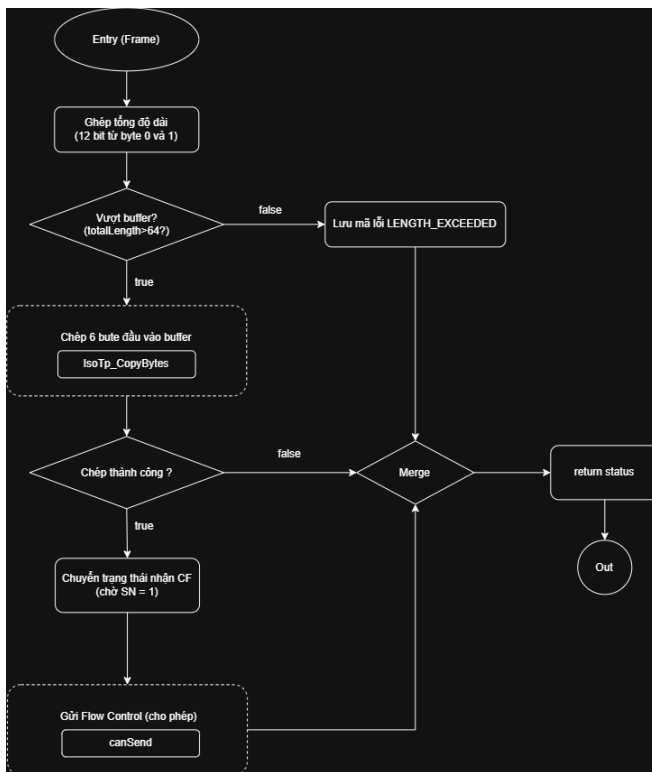


IsoTp_SendFirstFrame

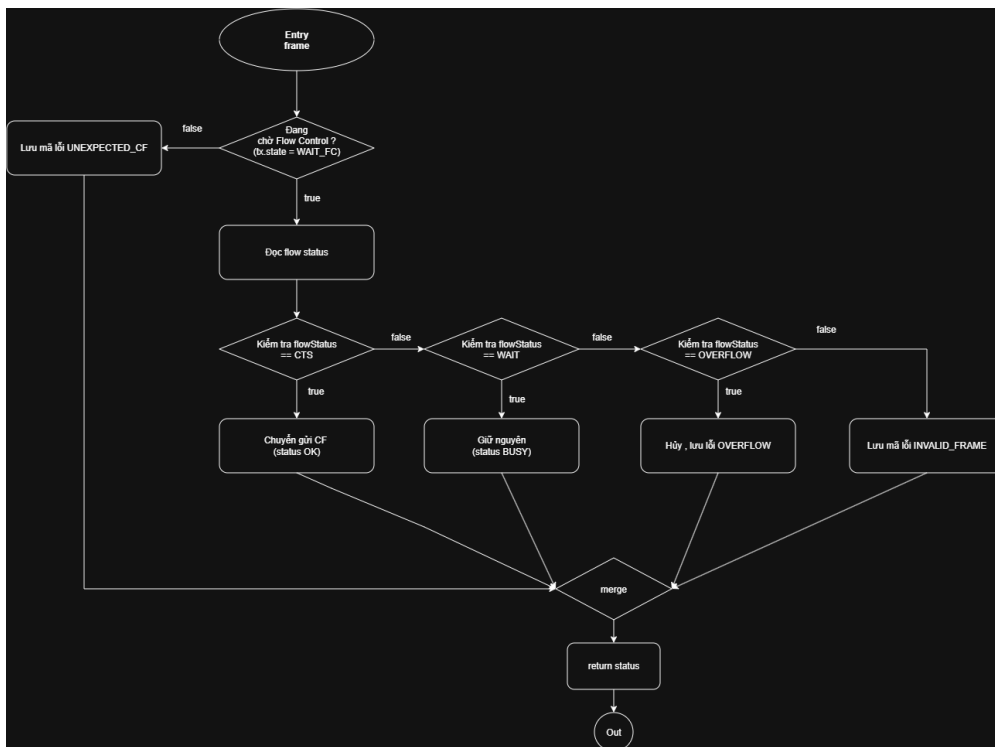
Phía nhận, *IsoTp_OnCanFrame()* phân loại 4 bit cao của byte PCI rồi gọi đúng handler tương ứng:



IsoTp_OnCanFrame: phân loại frame theo PCI và định tuyến tới handler



IsoTp_HandleReceivedFirstFrame: theo thiết kế, hàm này gửi Flow Control sau khi nhận First Frame



IsoTp_HandleReceivedFlowControl: đọc Block Size/STmin, cho phép gửi tiếp CF khi nhận CTS

Lưu ý: code hiện tại đã khác thiết kế ban đầu

Hai lưu đồ IsoTp_HandleReceivedFirstFrame và IsoTp_HandleReceivedFlowControl ở trên là thiết kế ban đầu, làm theo đúng ISO 15765-2. Lúc test, hai board không nhận được Flow Control ổn định nên nhóm đã tắt hẳn cơ chế này để chương trình chạy được, thay vì tìm cho ra nguyên nhân. Dấu vết trong mã nguồn hiện tại:

- isotp.h dòng #define ISOTP_USE_FLOW_CONTROL (0u) - cờ biên dịch tắt Flow Control cho toàn module.
- Trong IsoTp_HandleReceivedFirstFrame(): khi cờ này = 0, code KHÔNG gọi IsoTp_SendFlowControl() nữa, mà chuyển thẳng sang trạng thái ISOTP_RX_RECEIVING_CF ngay sau First Frame.
- Trong IsoTp_Send(): khi cờ này = 0, trạng thái TX bỏ qua ISOTP_TX_WAIT_FC, chuyển thẳng sang ISOTP_TX_SENDING_CF - bên gửi phát toàn bộ Consecutive Frame liên tiếp mà không chờ xác nhận Continue-To-Send từ bên nhận.
- Hàm IsoTp_HandleReceivedFlowControl() vẫn còn trong code nhưng thực chất là dead code khi cờ tắt: nó chỉ return ISOTP_OK ngay lập tức mà không đọc Block Size/STmin.

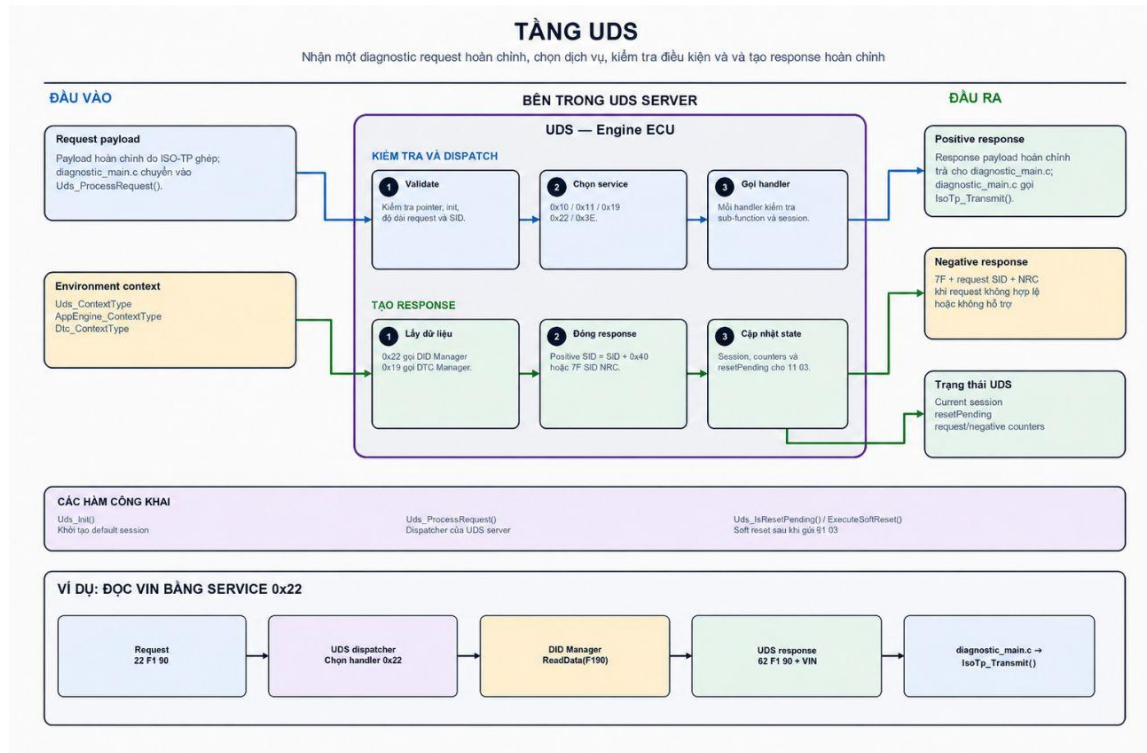
Vì vậy tầng ISO-TP bây giờ chỉ còn là cơ chế chia nhỏ bản tin một chiều: gửi First Frame rồi xả hết Consecutive Frame, không hỏi bên nhận có kịp xử lý hay không. Với bản tin ngắn như VIN thì vẫn chạy được, vì hai board đặt cạnh nhau trên bus 500 kbps không có tình huống nghẽn thật. Nhưng nếu ghép với một ECU thật cần yêu cầu tạm dừng hoặc báo tràn bộ đệm thì sẽ không chạy đúng.



Các mã lỗi nội bộ (NULL_POINTER, LENGTH_EXCEEDED, SF_BAD_LENGTH, SEQUENCE_MISMATCH, UNEXPECTED_CF, FC_OVERFLOW, TX_TIMEOUT, RX_TIMEOUT, NOT_INITIALISED, SEND_FAILED, UNEXPECTED_FC) vẫn được định nghĩa đầy đủ trong isotp.h, nhưng các nhánh liên quan tới Flow Control (FC_OVERFLOW, UNEXPECTED_FC) sẽ không còn phát sinh trong thực tế vì luồng FC đã bị tắt.

5.3. Tầng UDS

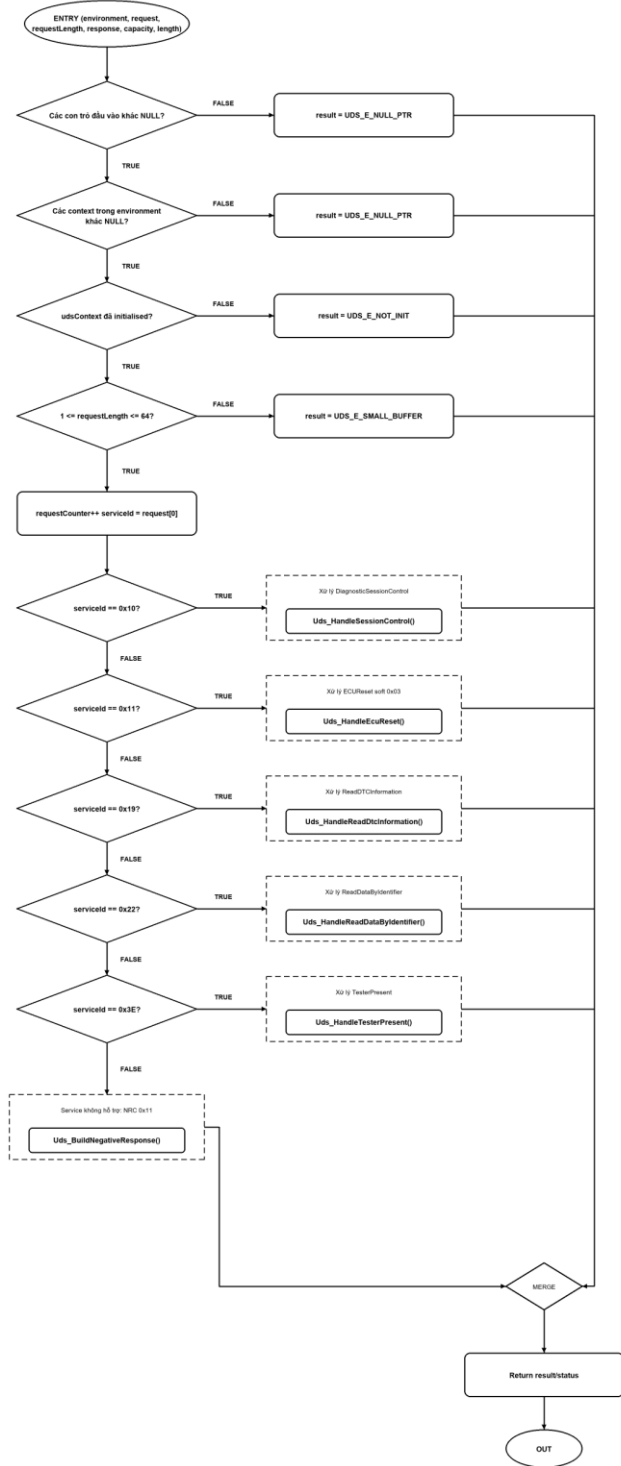
Tầng UDS cũng có detailed design đầy đủ cho từng service handler. `Uds_ProcessRequest()` là dispatcher trung tâm: kiểm tra request, tăng requestCounter, đọc Service ID (byte đầu) rồi gọi đúng handler.



Tổng quan tầng UDS: nhận request đã ghép, dispatch, tạo response

Uds_ProcessRequest

Validate request hoàn chỉnh, tăng requestCounter và phân phối theo Service ID.



Uds standard design | Orthogonal connection | Single exit point

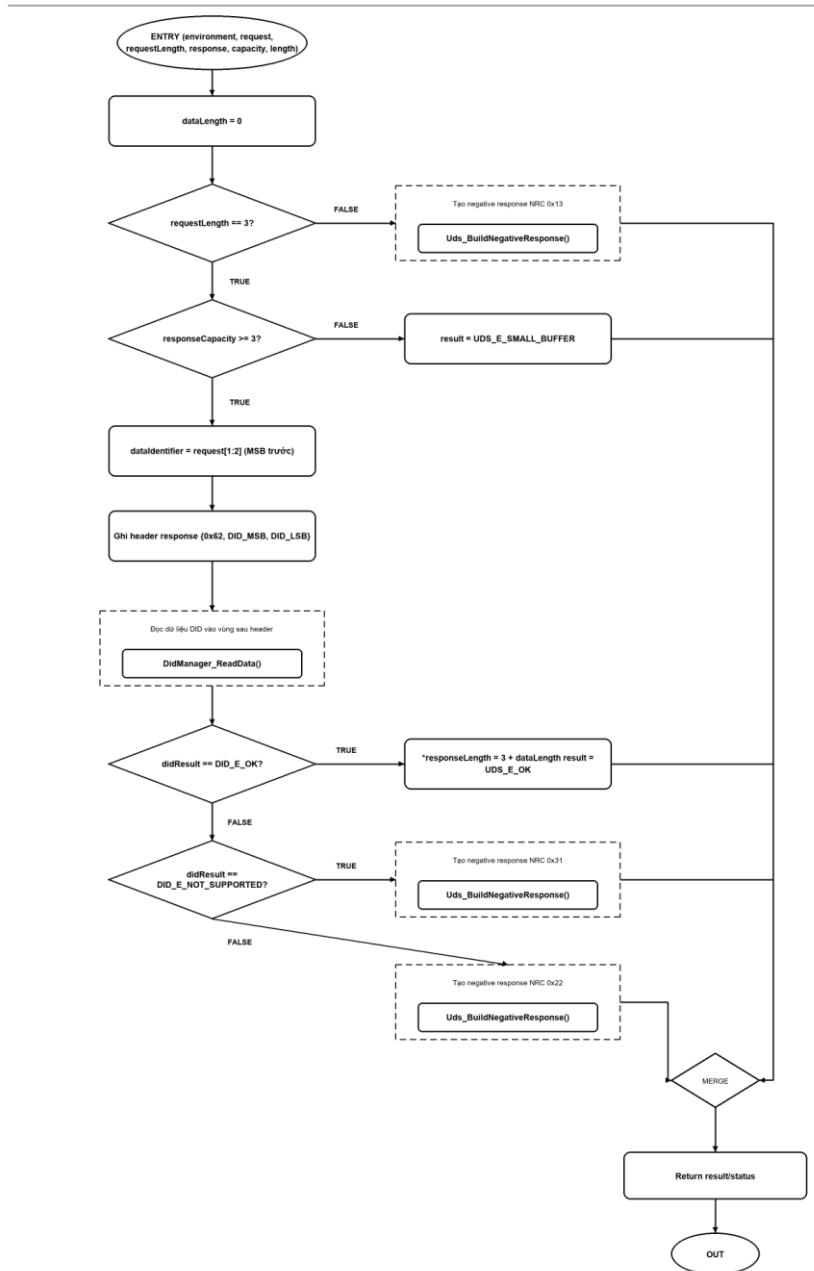
Uds_ProcessRequest: validate và phân phối theo Service ID

SID	Dịch vụ	Handler
0x10	DiagnosticSessionControl	Uds_HandleSessionControl()
0x11	ECUReset	Uds_HandleEcuReset()

SID	Dịch vụ	Handler
0x19	ReadDTCInformation	Uds_HandleReadDtcInformation()
0x22	ReadDataByIdentifier	Uds_HandleReadDataByIdentifier()
0x3E	TesterPresent	Uds_HandleTesterPresent()

Uds_HandleReadDataByIdentifier

Xử lý service 0x22: ghép DID, gọi DID Manager và tạo positive/negative response.



UDS detailed design | Orthogonal connectors | Single exit point

Uds_HandleReadDataByIdentifier: ghép DID, gọi DidManager_ReadData(), tạo positive/negative response

Đây là service duy nhất nhóm xác nhận hoạt động đúng chức năng trong thực tế: request 22 F1 90 trả về đúng 62 F1 90 kèm VIN qua UART log, đối chiếu khớp với thiết kế trên.

Uds_HandleSessionControl

Xử lý service 0x10: chỉ hỗ trợ Default Session 0x01 và Extended Session 0x03.

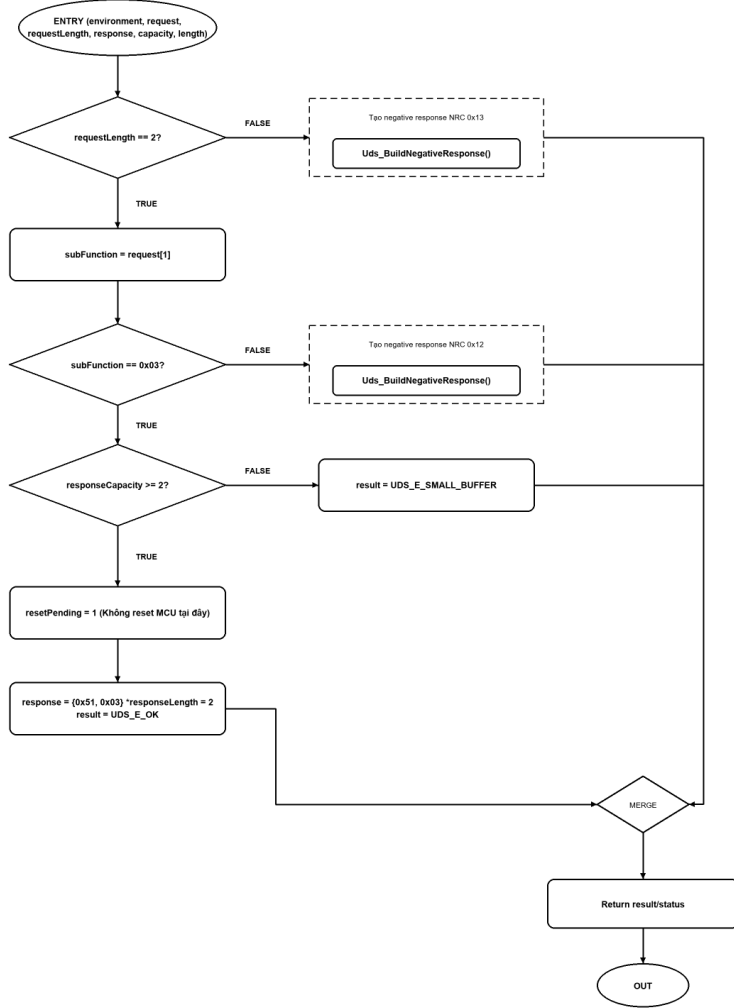


UDS detailed design | Orthogonal connectors | Single exit point

Uds_HandleSessionControl: chỉ hỗ trợ Default Session 0x01 và Extended Session 0x03

Uds_HandleEcuReset

Xử lý service 0x11; dự án chỉ hỗ trợ soft reset 0x03 và chỉ đặt cờ resetPending.

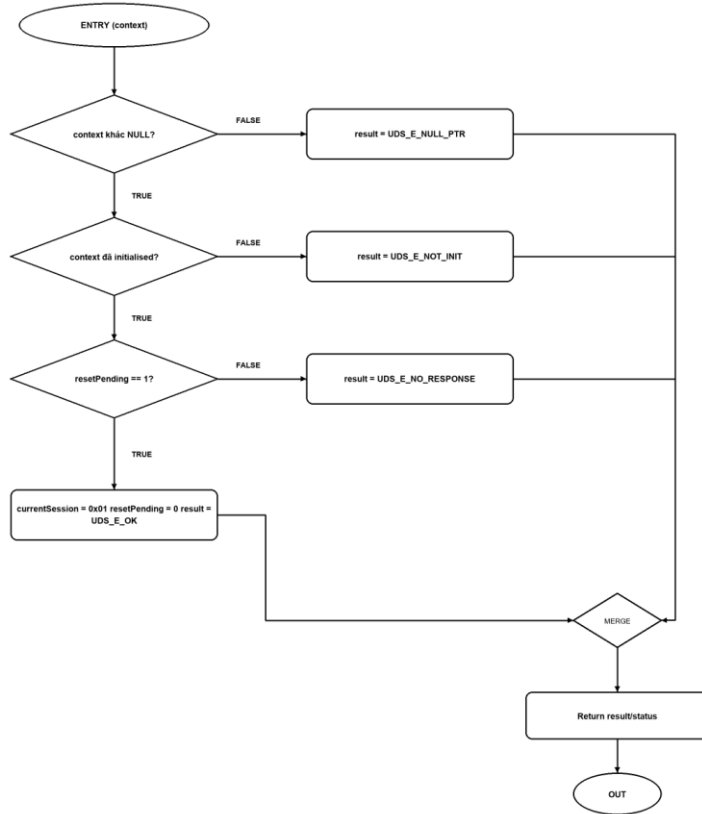


UDS detailed design | Orthogonal connectors | Single exit point

Uds_HandleEcuReset: chỉ hỗ trợ soft reset 0x03, đặt cờ resetPending = 1 (không reset MCU tại đây)

Uds_ExecuteSoftReset

Thực hiện reset phần mềm do dự án định nghĩa; không reset MCU hoặc phần cứng CAN.



UDS detailed design | Orthogonal connectors | Single exit point

Uds_ExecuteSoftReset: đặt lại currentSession = 0x01, resetPending = 0

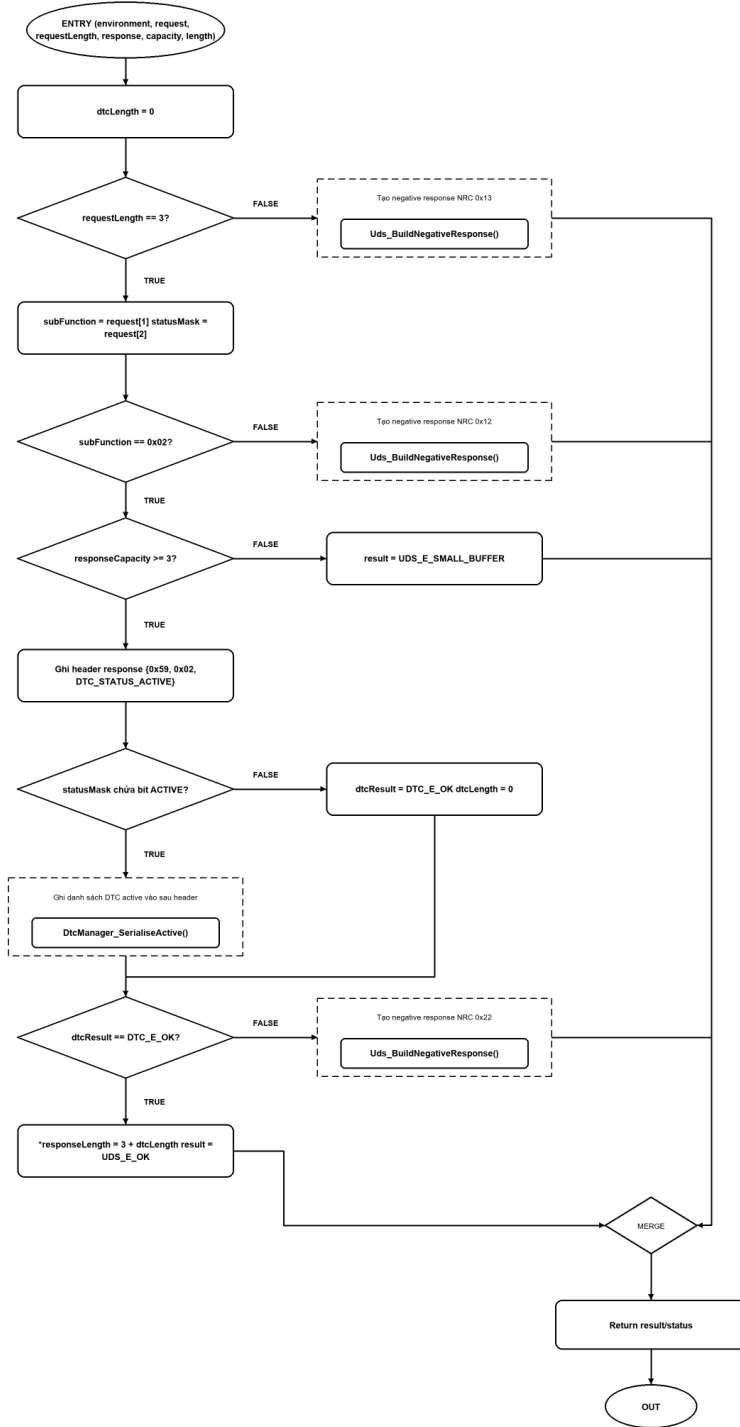
Lưu ý: hai dịch vụ này chưa chạy đúng chức năng

Ba lưu đồ Session Control, ECU Reset và Execute Soft Reset khớp với mã nguồn, nhưng phần việc phía sau thì chưa xong. Hiện nhóm mới in được log gửi - nhận ra Serial, chưa kiểm tra được hành vi cuối có đúng như mong đợi hay không:

- Session Control (0x10): việc chuyển currentSession sang 0x03 có thực thi trong code, nhưng chưa có cơ chế nào trong dự án thực sự khóa/mở các dịch vụ khác theo session hiện tại - nói cách khác, session được lưu nhưng không có tác dụng kiểm soát quyền truy cập nào cả.
- ECU Reset (0x11): Uds_ExecuteSoftReset() chỉ đặt lại hai biến phần mềm (currentSession, resetPending) - không có lệnh gọi NVIC_SystemReset(), watchdog reset hay bất kỳ cơ chế reset phần cứng nào. Đối chiếu main_EC.c/main_diag.c: không tồn tại lệnh reset MCU nào trong toàn bộ mã nguồn. Do đó "ECU Reset" hiện tại là reset trạng thái phần mềm giả lập, không phải reset thật.
- Cả hai luồng trên chưa có ca test end-to-end nào được thực thi và ghi log evidence - xem mục 7.

Uds_HandleReadDtcInformation

Xử lý service 0x19/0x02: đọc các DTC active theo status mask.



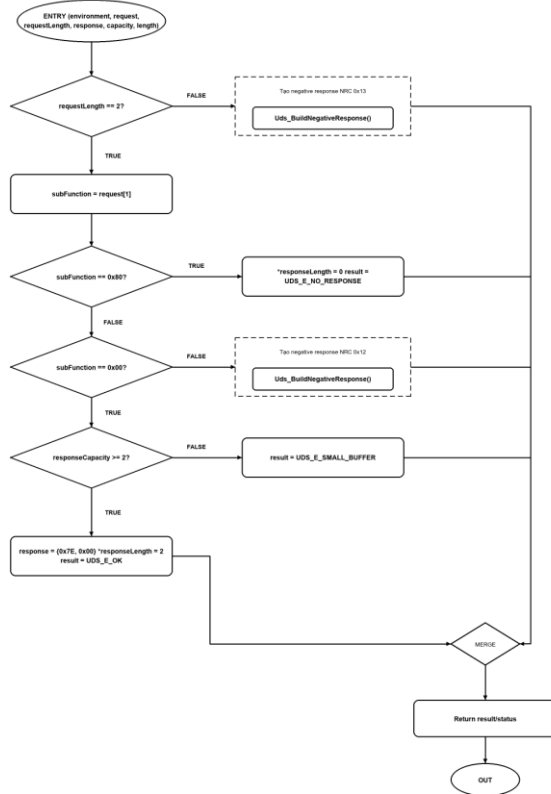
UDS detailed design | Orthogonal connectors | Single exit point

Uds_HandleReadDtcInformation: đọc danh sách DTC active theo status mask

Dịch vụ 0x19 gọi đúng DtcManager_SerialiseActive() như thiết kế, nhưng vì chưa tạo được lỗi active (mục 5.4) nên nó luôn trả về danh sách mặc định. Nhóm chưa lần nào thử được với một DTC thật đang bật.

Uds_HandleTesterPresent

Xử lý service 0x0E: trả lời 0x00 hoặc im lặng với sub-function 0x80.

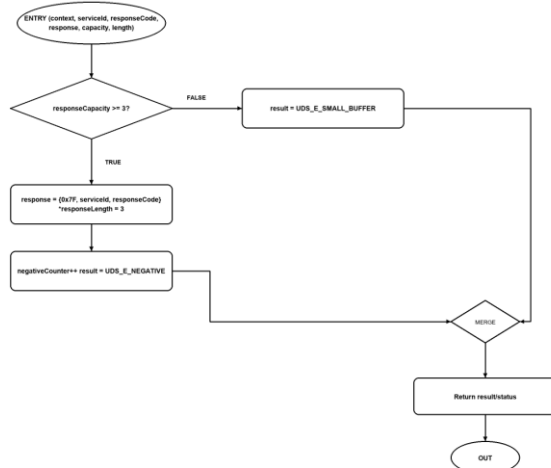


Uds module design | Orthogonal connectors | Single exit point

Uds_HandleTesterPresent: trả 7E 00 hoặc im lặng với sub-function 0x80

Uds_BuildNegativeResponse

Tạo negative response 0x7F, lưu SID/NRC và tăng bộ đếm lỗi.



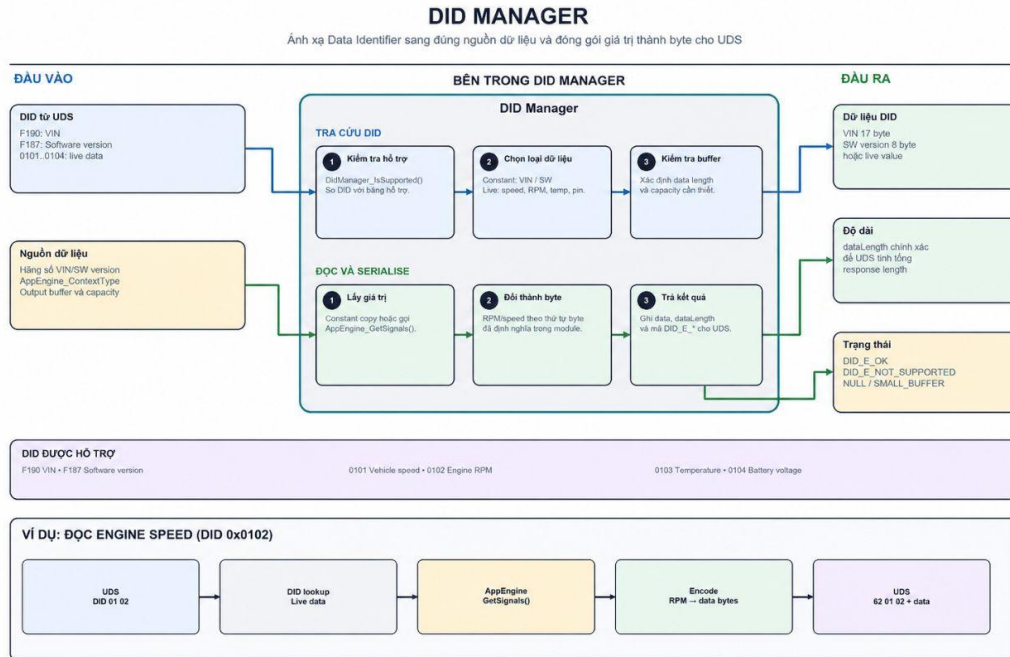
Uds module design | Orthogonal connectors | Single exit point

Uds_BuildNegativeResponse: dựng 7F + SID + NRC, tăng negativeCounter

Ba hàm phụ trợ `Uds_GetCurrentSession()`, `Uds_IsResetPending()` và `Uds_ClearResetPending()` chỉ đọc/ghi các cờ trạng thái nội bộ, không có nhánh xử lý phức tạp nên không trình bày lưu đồ riêng.

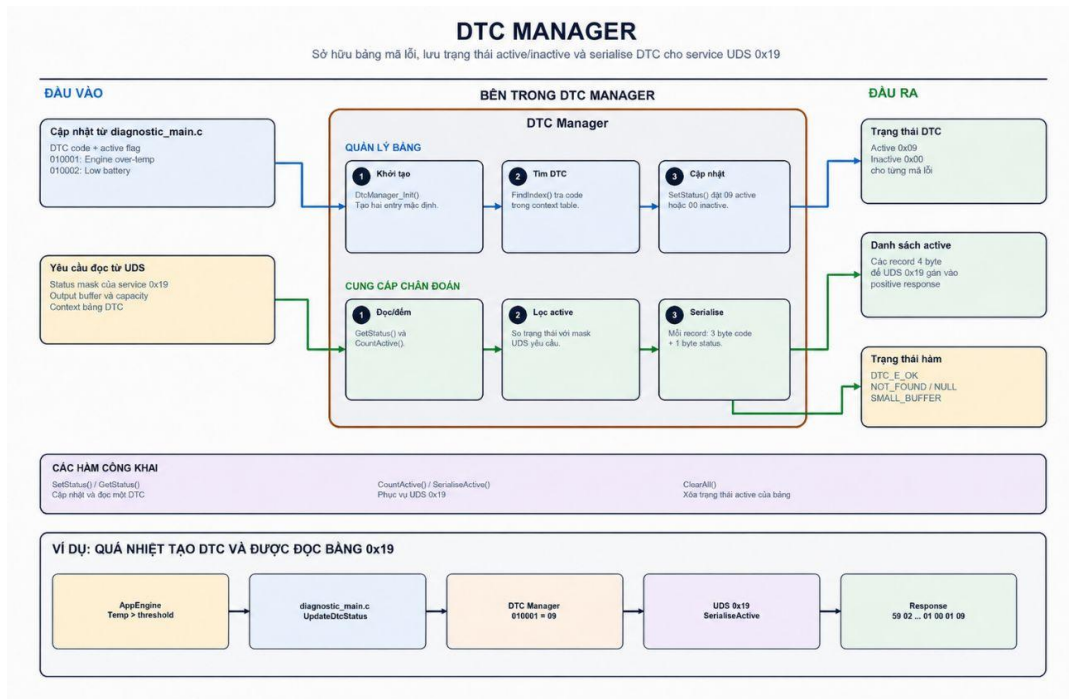
5.4. DID Manager, DTC Manager, App Engine, App Diag

Bốn module này chỉ có sơ đồ tổng quan (input/output ở mức module), không có lưu đồ chi tiết từng hàm như ISO-TP và UDS.



DID Manager: trả cứu DID, đọc/đóng gói dữ liệu VIN, SW version, 4 DID động

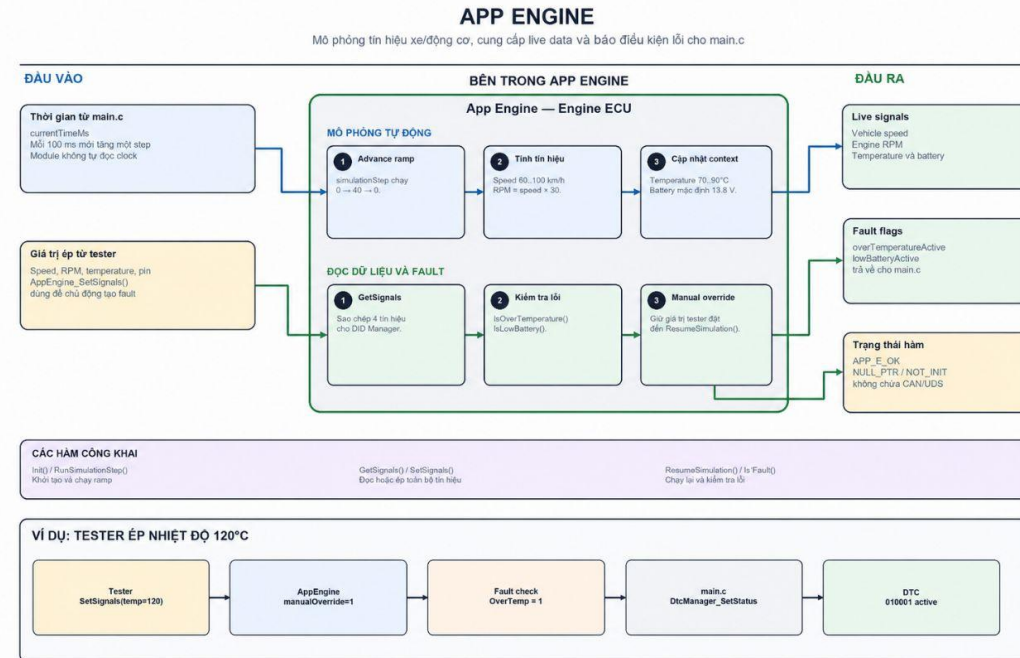
DID Manager hoạt động đúng với hai DID tĩnh (VIN, Software Version) và đã được xác nhận qua log UART. Bốn DID động (0x0101..0x0104) gọi AppEngine_GetSignals() để lấy dữ liệu mô phỏng - về code thì tồn tại, nhưng nhóm chưa xác nhận đã test riêng từng DID động này.



DTC Manager: quản lý bảng DTC, set/clear active, serialise cho service 0x19

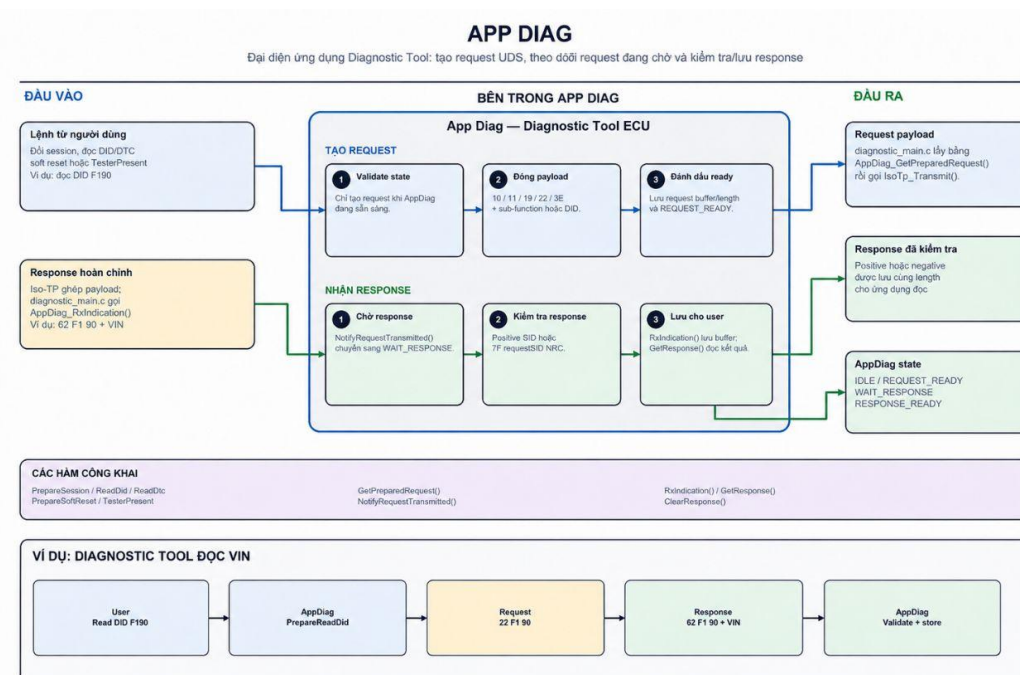
Lưu ý: chưa tạo được lỗi để đưa vào bảng DTC

Theo thiết kế, khi App Engine thấy nhiệt độ vượt ngưỡng hoặc điện áp thấp thì vòng lặp chính phải gọi `DtcManager_SetStatus()` để bật một DTC lên active. Đường gọi này chưa được nối trong code: không có lời gọi `DtcManager_SetStatus()` nào xuất phát từ điều kiện lỗi thật trong `main_EC.c` hay `app_engine.c`. Vì vậy bảng DTC giữ nguyên hai bản ghi mặc định suốt thời gian chạy, không phản ứng với điều kiện vận hành nào.



App Engine: mô phỏng tín hiệu động cơ, kiểm tra điều kiện lỗi, cho phép tester ép giá trị

App Engine có hàm `IsOverTemperature()/IsLowBattery()` và `AppEngine_SetSignals()` để tester ép giá trị (ví dụ ép nhiệt độ = 120°C), đúng như trong sơ đồ - nhưng như đã nêu ở trên, kết quả kiểm tra điều kiện lỗi này chưa được nối sang DTC Manager trong vòng lặp chính, nên việc ép giá trị qua UART chưa dẫn tới DTC active thực tế.



App Diag: tạo request UDS từ lệnh người dùng, theo dõi và kiểm tra response

App Diag đóng vai trò tester: nhận lệnh số (0-9) từ UART, tạo request UDS tương ứng, gửi qua IsoTp_Transmit(), rồi in response nhận được ra UART.

6. GIAO DIỆN VÀ KỊCH BẢN DEMO

6.1. Thiết kế giao diện

Người dùng điều khiển tester qua một menu văn bản in ra cổng Serial ở 115200 baud. Menu chính gồm năm dịch vụ chẩn đoán và lựa chọn thoát; mỗi phím số tương ứng một dịch vụ UDS, sau khi thực hiện xong màn hình trở lại menu để tiếp tục truy vấn.

```
=====
      STM32 DIAGNOSTIC TESTER
=====

===== DIAGNOSTIC MENU =====
1. Diagnostic Session Control
2. ECU Reset
3. Read DTC Information
4. Read Data by Identifier
5. Tester Present
6. Quit
Select:
Received: 1
Received HEX: 0x31

===== DIAGNOSTIC SESSION CONTROL =====
1. Default Session
2. Extended Session
3. Back
```

Hình 25. Giao diện menu chính trên Diagnostic Tester

Phím	Chức năng
1	Diagnostic Session Control
2	ECU Reset
3	Read DTC Information
4	Read Data by Identifier
5	Tester Present
6	Quit

6.2. Giao diện Read Data by Identifier

Khi chọn phím 4, tester chuyển sang menu con. Mục Read EngineStatus hiển thị dữ liệu chu kỳ đã nhận được; các mục còn lại tạo request UDS 0x22 theo DID tương ứng. Log in ra cả chuỗi byte gửi và nhận nên có thể đối chiếu trực tiếp với thiết kế.

```
[DIAG] TX: 22 01 04
[DIAG] RX: 62 01 04 8A
Battery Voltage: 13.8 V

===== READ DATA BY IDENTIFIER =====
1. Read EngineStatus
2. Read VIN
3. Read SW Info
4. Read Vehicle Speed
5. Read Engine RPM
6. Read Engine Temp
7. Read Battery Voltage
8. Back
Select:
Received: 8
Received HEX: 0x38

===== DIAGNOSTIC MENU =====
1. Diagnostic Session Control
2. ECU Reset
3. Read DTC Information
4. Read Data by Identifier
5. Tester Present
6. Quit
```

Hình 26. Menu Read Data by Identifier và kết quả đọc dữ liệu

Phím	Lựa chọn	Nguồn dữ liệu
1	Read EngineStatus	CAN ID 0x100
2	Read VIN	DID 0xF190
3	Read SW Info	DID 0xF187
4	Read Vehicle Speed	DID 0x0101
5	Read Engine RPM	DID 0x0102
6	Read Engine Temp	DID 0x0103
7	Read Battery Voltage	DID 0x0104
8	Back	Trở về menu chính

6.3. Video demo

Toàn bộ phiên demo trên hai board thật được quay lại và có thể xem tại đây: [Video demo hệ thống chẩn đoán UDS - Nhóm 1 \(Google Drive\)](#).

Video ghi lại lần lượt: cấp nguồn hai board và banner khởi động, đọc VIN và Software Version, đọc các DID dữ liệu động, thử một DID không tồn tại để thấy negative response, rồi tới các dịch vụ Session Control, ECU Reset và Read DTC.

6.4. Kịch bản demo và kết quả thực tế

Bảng dưới đây ghi lại từng thao tác trong video kèm kết quả quan sát được, tách rõ phần đã chạy đúng và phần mới chỉ trả về đúng khuôn dạng bản tin.

Thao tác	Kết quả quan sát	Nhận xét
Read VIN (22 F1 90)	62 F1 90 + VIN	Chạy đúng thiết kế
Read SW Info (22 F1 87)	62 F1 87 + chuỗi phiên bản	Chạy đúng thiết kế
Read Vehicle Speed / RPM / Temp / Battery	62 01 0x + giá trị mô phỏng	Đúng khuôn dạng; chưa kiểm tra riêng từng DID theo thời gian
DID không tồn tại (22 FF FF)	7F 22 31	Chạy đúng thiết kế
Diagnostic Session Control (10 03)	50 03	Đúng khuôn dạng; session chưa kiểm soát quyền truy cập
ECU Reset (11 03)	51 03	Đúng khuôn dạng; ECU không khởi động lại thật
Tester Present (3E 00)	7E 00	Đúng khuôn dạng
Read DTC (19 02 FF)	Danh sách DTC mặc định	Chưa tạo được lỗi active nên chưa kiểm tra được kịch bản DTC

Lưu ý về phần đọc DTC

Kịch bản đọc DTC trong video là kịch bản mong đợi theo thiết kế. Trên mã nguồn hiện tại, App Engine chưa được nối với DtcManager_SetStatus() nên việc ép quá nhiệt hay điện áp thấp không làm DTC chuyển sang active - phần này chưa tái hiện được trên phần cứng.

7. TEST REPORT

Truy cập test_report theo đường dẫn sau:

https://docs.google.com/spreadsheets/d/19jzVgMsNcssWtqEwmzYa8nj0Epx-lsSLuP8-a_Xv1ew/edit?usp=sharing

8. KẾT LUẬN

8.1. Kết quả đạt được

- Hoàn thiện tầng CanIf: khởi tạo CAN1, truyền/nhận frame có kiểm tra tham số đầy đủ, đây là tầng duy nhất nhóm xác nhận đạt chất lượng production trong phạm vi dự án.
- Bring-up thành công hai board STM32 giao tiếp CAN 500 kbps qua transceiver ngoài, có bằng chứng phần cứng.
- ISO-TP hoạt động ở mức phân mảnh cơ bản (Single Frame và First Frame/Consecutive Frame không kiểm soát luồng) đủ để truyền một bản tin ngắn như VIN.
- UDS dispatcher định tuyến đúng theo Service ID, dịch vụ ReadDataByIdentifier cho DID tĩnh (VIN, Software Version) hoạt động đúng như thiết kế và có log UART làm bằng chứng.
- Toàn bộ tài liệu V-Model (requirements, design, test spec) được viết đầy đủ, dù mức độ bám sát giữa tài liệu và code cài đặt thực tế còn khoảng cách lớn ở nhiều tầng.

8.2. Đánh giá quá trình làm việc nhóm

Phần này nhóm tự đánh giá lại cách làm việc của mình, viết ra để rút kinh nghiệm cho những lần sau chứ không nhằm chỉ trích ai.

- Nhóm hoạt động chưa hiệu quả và chưa có sự phối hợp chặt chẽ trong suốt quá trình làm việc: các tầng CanIf, ISO-TP, UDS, DID/DTC Manager được các thành viên khác nhau phát triển tương đối độc lập, thiếu các buổi đồng bộ giao diện (API) giữa các tầng trước khi bắt tay code, dẫn tới việc tích hợp cuối kỳ phát sinh nhiều vấn đề phải vá gấp.
- Quyết định tắt hoàn toàn Flow Control trong ISO-TP khi gặp lỗi lúc test là một ví dụ rõ cho việc thiếu quy trình xử lý sự cố: thay vì họp nhóm phân tích nguyên nhân gốc (ví dụ thời gian phản hồi FC, cấu hình timeout N_Bs, thứ tự gọi hàm giữa hai board), thành viên phụ trách đã tự sửa code theo hướng loại bỏ tính năng để "chạy được", làm mất đi giá trị kỹ thuật cốt lõi của tầng ISO-TP mà không có review hoặc thông báo cho các phần liên quan.
- Tiến độ giữa các thành viên không đồng đều: tầng CanIf hoàn thiện sớm và kỹ, trong khi Session Manager, DTC injection, ECU Reset thật gần như chưa được bắt đầu triển khai đến cùng - cho thấy thiếu việc theo dõi tiến độ chung và phân bổ lại nguồn lực khi có thành viên chậm.
- Thiếu một đầu mối kỹ thuật (technical lead) chốt các quyết định thiết kế xuyên tầng, dẫn tới tài liệu design (ví dụ sơ đồ Flow Control) và code thực tế lệch nhau mà không ai cập nhật lại tài liệu hoặc cảnh báo cho cả nhóm.

8.3. Hạn chế kỹ thuật

- ISO-TP không còn tuân thủ đầy đủ ISO 15765-2 do Flow Control bị vô hiệu hóa hoàn toàn (ISOTP_USE_FLOW_CONTROL = 0) - đây là hạn chế kỹ thuật nghiêm trọng nhất, khiến tầng này chỉ còn ý nghĩa như một cơ chế phân mảnh đơn giản.
- Session Control chưa có tác dụng kiểm soát quyền truy cập dịch vụ - mọi dịch vụ đều xử lý được bất kể session hiện tại là gì.
- DTC Manager chưa có cơ chế inject lỗi từ điều kiện vận hành thật (quá nhiệt, điện áp thấp) sang bảng DTC - tính năng chẩn đoán lỗi cốt lõi của UDS chưa hoạt động.
- ECU Reset (dịch vụ 0x11) không thực hiện reset phần cứng thật, chỉ đặt lại hai biến phần mềm.

- Phần lớn việc "kiểm thử" mới ở mức log UART quan sát bằng mắt, chưa có công cụ đo độc lập (CAN analyzer, oscilloscope) hay test tự động.
- Dự án mới thật sự hoàn thành chắc chắn ở tầng CanIf; các tầng cao hơn (ISO-TP, UDS, DID/DTC Manager) đều còn tồn tại khoảng cách giữa thiết kế và hành vi thực tế đã kiểm chứng.

9. HƯỚNG PHÁT TRIỂN

- Khôi phục Flow Control đúng chuẩn ISO 15765-2: tìm nguyên nhân gốc khiến FC không nhận được ổn định (kiểm tra lại timing N_Ar/N_As, thứ tự IsoTp_MainFunction() trong vòng lặp, cấu hình ngắt CAN) trước khi quyết định tắt tính năng.
- Hoàn thiện Session Manager thực sự kiểm soát quyền truy cập dịch vụ theo session hiện tại (ví dụ: một số dịch vụ chỉ chạy được ở Extended Session).
- Nối luồng phát hiện lỗi ở App Engine (IsOverTemperature/IsLowBattery) với DtcManager_SetStatus() trong vòng lặp chính để DTC injection hoạt động thật.
- Thực hiện ECU Reset thật bằng NVIC_SystemReset() hoặc watchdog reset, có cơ chế gửi response 51 03 trước khi reset để tester nhận được xác nhận.
- Thiết lập lại quy trình làm việc nhóm: họp đồng bộ API giữa các tầng trước khi code, review code bắt buộc trước khi merge (đặc biệt với các thay đổi ảnh hưởng thiết kế như việc tắt Flow Control), và một đầu mối kỹ thuật theo dõi tiến độ chung.

